Apostila: API REST Acadêmica v2

Engenharia de Software — PHP Puro sem Frameworks

Sumário

1. Visão Geral do Projeto
2. Formas de Autenticação e a Escolha do Projeto
3. Estrutura de Pastas
    - 3.1 Subpastas de `app/`
4. Classes do Projeto — Explicação e Exemplos
5. Padrão Swagger e o swagger-ui.html
6. 50 Perguntas sobre o Projeto
7. Gabarito Comentado

1. Visão Geral do Projeto

O que é esta API?

Este projeto é uma **API REST acadêmica** construída em **PHP puro** (sem frameworks como Laravel ou Symfony). O objetivo é demonstrar, de forma didática e estruturada, como construir uma API profissional do zero, aplicando boas práticas de arquitetura de software.

A API gerencia **clientes** e protege suas rotas com **autenticação JWT** (JSON Web Token). É um sistema completo de cadastro que inclui:

- Autenticação com usuário e senha
- Controle de sessão via tokens (access token + refresh token)
- CRUD de clientes com validação de CPF brasileiro
- Documentação interativa via Swagger UI

Por que sem framework?

Em ambiente acadêmico, usar um micro-framework de zero força o estudante a entender **como as coisas funcionam por baixo dos panos**:

- Como um roteador analisa URLs e despacha requisições
- Como um autoloader mapeia classes para arquivos
- Como variáveis de ambiente são carregadas
- Como o JWT é gerado e validado manualmente
- Como o padrão MVC é implementado sem magia

Pilha tecnológica

Camada	Tecnologia

Camada	Tecnologia
Linguagem	PHP 8.1+
Banco de dados	MySQL / MariaDB 5.7+
Servidor web	Apache com mod_rewrite
Autenticação	JWT HS256 + Refresh Token Rotation
Documentação	OpenAPI 3.0 + Swagger UI
Ambiente	XAMPP (desenvolvimento local)

Endpoints disponíveis

Método	Rota	Proteção	Descrição
POST	`/auth/login`	Pública	Autentica usuário e retorna tokens
POST	`/auth/refresh`	Pública	Renova tokens com refresh token
GET	`/clientes`	JWT	Lista todos os clientes
POST	`/clientes`	JWT	Cadastra novo cliente
GET	`/clientes/{id}`	JWT	Busca cliente por ID
PUT	`/clientes/{id}`	JWT	Atualiza cliente
DELETE	`/clientes/{id}`	JWT	Remove cliente
GET	`/docs`	Pública	Interface Swagger UI
GET	`/docs/openapi.json`	Pública	Especificação OpenAPI

Fluxo geral de uma requisição

```
Cliente HTTP (cURL, Postman, frontend)
        |
        | HTTP Request
        ↓
  [.htaccess raiz]  ←── redireciona tudo para /public/
        |
        ↓
  [public/.htaccess] ←── redireciona tudo que não é arquivo para index.php
        |
        ↓
  [public/index.php]
        | → carrega variáveis de ambiente (.env)
        | → registra autoloader PSR-4
        | → configura cabeçalhos CORS
        | → instancia Request e Response
        | → carrega as rotas (routes/api.php)
        | → chama Router::despachar()
```

```
        |
        ↓
[Router] ←── encontra a rota correspondente
        | → executa middlewares (se houver)
        | → chama o Controller
        |
        ↓
[Controller] ←── recebe Request, devolve Response
        | → valida dados básicos
        | → chama o Service
        |
        ↓
[Service] ←── contém a lógica de negócio
        | → valida regras de negócio
        | → chama o Model
        |
        ↓
[Model] ←── acessa o banco de dados via PDO
        | → retorna dados ao Service
        ↑
        | (resposta sobe a cadeia)
        ↓
Response::send() ←── envia JSON ao cliente
```

---

2. Formas de Autenticação e a Escolha do Projeto

O problema: como saber quem está fazendo a requisição?

O protocolo HTTP é **stateless** — cada requisição é independente e o servidor não "lembra" de requisições anteriores. Isso significa que, a cada chamada, o cliente precisa provar sua identidade.

Existem várias abordagens para resolver isso:

2.1 HTTP Basic Authentication

O cliente envia `usuario:senha` codificado em Base64 no cabeçalho de cada requisição.

```
Authorization: Basic YWRtaW46MTIzNDU2
```

Vantagens:

- Simples de implementar
- Suportado nativamente por navegadores

Desvantagens:

- Envia credenciais em toda requisição (risco se não usar HTTPS)
- Sem controle de sessão (não dá para "expirar" o acesso)

- Base64 não é criptografia — qualquer pessoa com acesso ao tráfego vê o conteúdo

Uso adequado: Ferramentas internas simples, onde HTTPS é garantido e o risco é baixo.

2.2 Sessões no servidor (Session-based)

O servidor cria uma sessão e armazena no servidor (arquivo, Redis, banco). Retorna ao cliente um `session_id` via cookie.

```
Set-Cookie: PHPSESSID=abc123xyz; HttpOnly; Secure
```

Vantagens:

- Controle total no servidor (pode invalidar qualquer sessão)
- Bem estabelecido em aplicações web tradicionais

Desvantagens:

- **Stateful** — o servidor precisa armazenar estado, dificultando escala horizontal
- Problemático com múltiplos servidores (precisa de sessão compartilhada)
- Cookies não funcionam naturalmente em APIs consumidas por apps mobile ou outros servidores

Uso adequado: Aplicações web tradicionais com renderização server-side.

2.3 API Keys

O cliente recebe uma chave estática que identifica sua aplicação e a envia em cada requisição.

```
Authorization: ApiKey abc123def456
```

ou como query string:

```
GET /clientes?api_key=abc123def456
```

Vantagens:

- Simples de implementar e de usar
- Ideal para comunicação máquina-a-máquina

Desvantagens:

- Sem expiração automática
- Se vazar, fica comprometida até ser revogada manualmente
- Não identifica o usuário final, apenas a aplicação

Uso adequado: Integrações B2B, webhooks, SDKs que acessam API em nome de um serviço (não de um usuário).

2.4 OAuth 2.0

Protocolo de autorização que permite que uma aplicação acesse recursos em nome de um usuário, sem que o usuário precise compartilhar sua senha com a aplicação.

Fluxo simplificado (Authorization Code):

1. App redireciona usuário para servidor de autorização (Google, GitHub etc.)
2. Usuário autentica no servidor de autorização
3. Servidor retorna um `code` para a app
4. App troca o `code` por um `access_token`
5. App usa o `access_token` para acessar recursos

Vantagens:

- Padrão da indústria para autorização delegada
- O usuário nunca entrega sua senha para a aplicação terceira
- Suporte a múltiplos escopos e permissões granulares

Desvantagens:

- Complexo de implementar corretamente
- Requer um Authorization Server (ou usar um pronto como Auth0, Keycloak)
- Exagero para APIs internas simples

Uso adequado: "Login com Google/Facebook", plataformas que expõem API para terceiros.

2.5 JWT — JSON Web Token ✅ (escolha deste projeto)

O servidor gera um **token assinado** contendo as informações do usuário. O cliente armazena e envia esse token em cada requisição. O servidor **não precisa consultar o banco** para validar — a assinatura prova a autenticidade.

```
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsImV4cCI6MTcwMDAwMH0.signature
```

Estrutura do JWT:

```
HEADER.PAYLOAD.SIGNATURE
  ↑         ↑         ↑
Base64Url  Base64Url  HMAC-SHA256(header+payload, secret)
```

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload (claims):

```
{
  "iss": "api-academica",
  "iat": 1700000000,
  "exp": 1700000900,
  "sub": 1,
  "usuario": "admin"
}
```

Vantagens:

- **Stateless** — nenhum estado no servidor para validar o token
- Funciona perfeitamente para APIs REST e apps mobile
- Auto-contido — o payload carrega as informações do usuário
- Fácil de escalar horizontalmente

Desvantagens:

- Não pode ser invalidado antes da expiração (sem estado no servidor)
- Se o secret vazar, todos os tokens ficam comprometidos
- O payload é visível (apenas assinado, não criptografado)

2.6 Refresh Token Rotation (estratégia adotada)

Para compensar a desvantagem dos JWTs de curta duração, este projeto implementa **Refresh Token Rotation**:

1. No login, o servidor retorna **dois tokens**:
    - **Access Token** (JWT, curta duração: 15 minutos)
    - **Refresh Token** (string aleatória, longa duração: 7 dias, armazenado no banco)
2. Quando o access token expira, o cliente envia o refresh token para `/auth/refresh`
3. O servidor:
    - Busca o refresh token no banco de dados
    - Verifica se ainda é válido (não expirou)
    - **Invalida o refresh token antigo**
    - Gera um novo par de tokens (access + refresh)
    - Retorna ao cliente

```
Cliente                    Servidor
  |                           |
  |-- POST /auth/login -----→ |
  |← access_token (15min) --- |
  |← refresh_token (7 dias) - |
  |                           |
  |  [usa access_token] ...   |
  |                           |
  |  [access_token expirou]   |
  |                           |
  |-- POST /auth/refresh ---→ |
  |   {refresh_token: "..."}  |-- Invalida token antigo
  |← novo access_token ------ |-- Gera novo refresh token
  |← novo refresh_token ----- |-- Salva no banco
```

Por que invalidar o refresh token antigo?

Se um atacante roubar um refresh token e tentar usá-lo depois do usuário legítimo já ter feito refresh, o servidor detecta que o token já foi usado e pode bloquear ambos, protegendo o sistema.

---

3. Estrutura de Pastas

```
v2/
├── app/           → Código da aplicação (MVC)
├── config/        → Configurações (banco, JWT)
├── database/      → Scripts SQL e seed de dados
├── docs/          → Especificação OpenAPI
├── public/        → Único diretório exposto ao Apache
├── routes/        → Definição das rotas
├── .env           → Variáveis de ambiente (não commitar!)
├── .env.example   → Template do .env para equipe
└── .htaccess      → Redirecionamento raiz para /public/
```

`app/` — O coração da aplicação

Contém todo o código PHP da aplicação organizado por responsabilidade. **Nunca é acessado diretamente pelo Apache** — isso é uma medida de segurança. Os arquivos de configuração, modelos e lógica de negócio ficam protegidos.

`config/` — Configurações centralizadas

Arquivos PHP que retornam arrays de configuração. Separa os parâmetros configuráveis do código que os usa. Lê variáveis do `.env` via `Env::get()`, nunca hardcoda valores sensíveis.

Por que separar do código?

- Facilita mudança entre ambientes (dev, staging, produção)
- Evita espalhar valores mágicos pelo código
- Um único lugar para alterar uma configuração

`database/` — Scripts de banco de dados

Contém o esquema SQL (`migration.sql`) e o script de população inicial (`seed.php`). Separa a estrutura do banco do código da aplicação.

Por que separar?

- O DBA (ou outro desenvolvedor) pode trabalhar no banco sem mexer no código
- Facilita recriar o banco em qualquer ambiente
- Documenta a evolução do schema ao longo do tempo

`public/` — A única porta de entrada

Este é o único diretório que o Apache deve servir. Contém apenas:

- `index.php` — Front Controller (ponto de entrada único)
- `.htaccess` — Regras de reescrita de URL
- `swagger-ui.html` — Interface de documentação

Por que apenas esse diretório fica exposto?

Se o Apache servisse toda a pasta `v2/`, qualquer pessoa poderia acessar diretamente:

- `http://localhost/a/v2/config/database.php` → credenciais do banco
- `http://localhost/a/v2/.env` → variáveis de ambiente sensíveis
- `http://localhost/a/v2/app/Models/ClienteModel.php` → código-fonte

Ao expor apenas `/public/`, o Apache serve somente o que foi colocado lá intencionalmente.

`routes/` — Mapa da API

Define quais URLs existem, quais controllers respondem a elas e quais middlewares protegem cada uma. Centralizar as rotas aqui facilita visualizar toda a API de um só lugar.

Por que separar do index.php?

- O `index.php` cuida de bootstrapping (carregar dependências, configurar ambiente)
- O `api.php` cuida apenas de declarar rotas
- Cada arquivo tem uma única responsabilidade (princípio SRP)

3.1 Subpastas de `app/`

`app/Controllers/`

Responsabilidade: Receber a requisição HTTP, extrair os dados necessários, chamar o Service adequado e montar a resposta.

Regra de ouro: Controllers não contêm lógica de negócio. Eles são apenas "coordenadores" — leem a entrada, delegam o trabalho, escrevem a saída.

```
Requisição HTTP → Controller → Service → Model → Banco
                      ↑
              Apenas coordena o fluxo
```

Não deve fazer em um Controller:

- Validar CPF
- Calcular idades
- Formatar dados
- Acessar banco diretamente

`app/Core/`

Responsabilidade: Infraestrutura do micro-framework. Classes genéricas que não pertencem ao domínio do negócio (clientes, autenticação), mas que sustentam toda a aplicação.

Contém as peças que, em um framework real como Laravel, já vêm prontas:

- `Router` — o roteador
- `Request` — o objeto de requisição
- `Response` — o objeto de resposta
- `JWT` — utilitário de tokens
- `Env` — carregador de variáveis de ambiente

`app/Database/`

Responsabilidade: Gerenciar a conexão com o banco de dados. Implementa o padrão **Singleton** para garantir que apenas uma conexão PDO seja criada durante todo o ciclo de vida da requisição.

Por que Singleton aqui?

- Abrir uma conexão com banco é caro (TCP handshake, autenticação MySQL)
- Reutilizar a mesma conexão é muito mais eficiente
- Evita o risco de esgotar o pool de conexões do MySQL

`app/Middleware/`

Responsabilidade: Interceptar a requisição **antes** de chegar ao Controller para aplicar verificações transversais — coisas que valem para múltiplas rotas.

Exemplos de middlewares comuns:

- Autenticação (verifica JWT) ← implementado neste projeto
- Autorização (verifica permissões/roles)
- Rate limiting (limita requisições por IP)
- Logging (registra todas as requisições)
- CORS (cabeçalhos de origem cruzada)

O middleware retorna `true` para deixar a requisição prosseguir ou `false` (com resposta de erro) para bloqueá-la.

`app/Models/`

Responsabilidade: Toda a comunicação com o banco de dados. Cada Model representa uma entidade (tabela) e seus métodos são as operações possíveis sobre essa entidade (buscar, criar, atualizar, excluir).

Regra de ouro: Models não contêm lógica de negócio. Eles apenas traduzem chamadas PHP em queries SQL e retornam os dados.

```
-- O Model executa isso...
SELECT * FROM clientes WHERE id = ?

-- ...e retorna isso ao Service:
['id' => 1, 'nome' => 'Maria', 'cpf' => '529.982.247-25', ...]
```

`app/Services/`

Responsabilidade: Lógica de negócio. É aqui que vivem as regras que fazem a aplicação ter valor: validar CPF, garantir unicidade de dados, implementar refresh token rotation, etc.

A camada Service é o "cérebro" da aplicação:

- Recebe dados do Controller
- Aplica validações e regras de negócio
- Coordena chamadas a um ou mais Models
- Retorna o resultado ao Controller

Por que separar Service de Controller?

- Reutilização: a mesma lógica de negócio pode ser chamada de múltiplos controllers, comandos CLI ou jobs
- Testabilidade: Services podem ser testados sem simular HTTP
- Clareza: fica óbvio onde as regras do negócio estão

---

4. Classes do Projeto — Explicação e Exemplos

`Core/Env.php`

O que faz: Carrega o arquivo `.env` e disponibiliza as variáveis como configuração acessível em qualquer lugar da aplicação via `Env::get()`.

Por que existe: Colocar senhas, chaves e configurações diretamente no código é perigoso — elas acabam no repositório Git. O padrão `.env` resolve isso: o arquivo `.env` é ignorado pelo Git (`.gitignore`) e contém os valores reais; o código apenas lê variáveis.

Métodos:

```
// Carrega o arquivo .env
Env::carregar(string $caminho): void

// Lê uma variável (com valor padrão opcional)
Env::get(string $chave, mixed $padrao = null): mixed
```

Como é usado no projeto:

```
// public/index.php — chamado uma vez no bootstrap
Env::carregar(__DIR__ . '/../.env');

// config/database.php — lê as variáveis carregadas
return [
    'host'    => Env::get('DB_HOST', 'localhost'),
    'dbname'  => Env::get('DB_NAME', 'api_academica'),
    'usuario' => Env::get('DB_USER', 'root'),
    'senha'   => Env::get('DB_PASS', ''),
];

// config/jwt.php
return [
    'segredo'           => Env::get('JWT_SECRET'),
    'expiracao_acesso'  => (int) Env::get('JWT_ACCESS_EXPIRY', 900),
    'expiracao_refresh' => (int) Env::get('JWT_REFRESH_EXPIRY', 604800),
];
```

Arquivo `.env` correspondente:

```
DB_HOST=localhost
DB_NAME=api_academica
DB_USER=root
DB_PASS=
JWT_SECRET=minha-chave-secreta-longa
JWT_ACCESS_EXPIRY=900
JWT_REFRESH_EXPIRY=604800
APP_ENV=development
```

`Core/JWT.php`

O que faz: Gera e valida tokens JWT usando o algoritmo HS256 (HMAC-SHA256). Implementação do zero, sem bibliotecas externas.

Estrutura de um JWT:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9    ← Header (Base64Url)
.
eyJpc3MiOiJhcGktYWNhZGVtaWNhIiwic3ViIjoxfQ    ← Payload (Base64Url)
.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c   ← Assinatura (HMAC-SHA256)
```

Métodos:

```
// Gera um novo token JWT
JWT::gerar(array $payload, string $segredo): string

// Valida e decodifica um token (lança exceção se inválido)
JWT::validar(string $token, string $segredo): array
```

Como é usado no projeto:

```
// AuthService.php — ao fazer login, gera o access token
$config = require __DIR__ . '/../../config/jwt.php';
$payload = [
    'iss'     => 'api-academica',
    'iat'     => time(),
    'exp'     => time() + $config['expiracao_acesso'],  // +900 seg
    'sub'     => $usuario['id'],
    'usuario' => $usuario['usuario'],
];
$accessToken = JWT::gerar($payload, $config['segredo']);

// AuthMiddleware.php — ao receber uma requisição protegida
$config  = require __DIR__ . '/../../config/jwt.php';
$payload = JWT::validar($token, $config['segredo']);
// Se inválido, JWT::validar() lança InvalidArgumentException
// Se válido, $payload contém os dados do usuário
```

Por que `hash_equals()` na comparação de assinatura?

Comparações comuns de string (`===`) podem vazar informação de tempo: quanto mais caracteres corretos, mais tempo leva para falhar. O `hash_equals()` sempre leva o mesmo tempo independente da diferença — prevenindo **ataques de timing**.

Core/Request.php

O que faz: Encapsula todos os dados da requisição HTTP em um objeto organizado. Em vez de usar `$_SERVER`, `$_POST`, `php://input` espalhados pelo código, tudo fica em um único objeto.

Propriedades principais:

- `$metodo` — GET, POST, PUT, DELETE
- `$caminho` — URL sem a base (ex: `/clientes/5`)
- `$parametrosRota` — parâmetros extraídos da URL (ex: `['id' => '5']`)
- `$atributos` — dados injetados por middlewares
- `$corpo` — body da requisição (lazy-loaded)

Métodos:

```
getMetodo(): string           // "GET", "POST", "PUT", "DELETE"
getCaminho(): string          // "/clientes/5"
getHeader(string $nome): ?string  // lê cabeçalho HTTP
getCorpo(): array             // body JSON decodificado
getParam(string $nome): mixed // parâmetro da rota (:id, {id})
getAtributo(string $nome): mixed  // atributo setado por middleware
```

Como é usado no projeto:

```
// Router.php — ao fazer match de rota, injeta parâmetros
$request->setParametrosRota(['id' => '42']);

// AuthMiddleware.php — injeta dados do usuário autenticado
$request->setAtributo('usuario', $payload);

// ClienteController.php — lê o ID da URL
$id = (int) $request->getParam('id');

// ClienteController.php — lê o corpo da requisição
$dados = $request->getCorpo();
// $dados = ['cpf' => '529.982.247-25', 'nome' => 'Maria', ...]

// AuthMiddleware.php — lê o cabeçalho Authorization
$authHeader = $request->getHeader('Authorization');
// $authHeader = "Bearer eyJhbGciOiJIUzI1NiJ9..."
```

Core/Response.php

O que faz: Constrói e envia respostas HTTP. Centraliza a lógica de enviar JSON, definir status codes e cabeçalhos.

Propriedades principais:

- `$status` — código HTTP (200, 201, 400, 401, 404, 409, 500)
- `$headers` — array de cabeçalhos a enviar
- `$corpo` — conteúdo da resposta
- `$enviada` — flag que impede envio duplicado

Métodos:

```
json(array $dados, int $status = 200): self   // prepara resposta JSON
html(string $conteudo, int $status = 200): self // prepara resposta HTML
send(): void                                    // envia ao cliente
foiPreparada(): bool                            // verifica se foi preparada
```

Interface fluente (method chaining):

```
// Equivalente a:
$response->json(['erro' => 'Não encontrado'], 404);
$response->send();
```

Como é usado no projeto:

```
// AuthController.php — login bem-sucedido
$response->json([
    'status' => 'success',
    'data'   => $tokens,  // access_token, refresh_token, etc.
], 200);

// ClienteController.php — cliente criado
$response->json([
    'status' => 'success',
    'data'   => $cliente,
], 201);  // 201 Created

// ClienteController.php — cliente não encontrado
$response->json([
    'status'  => 'error',
    'message' => $e->getMessage(),
], 404);

// routes/api.php — servir o Swagger UI
$html = file_get_contents(__DIR__ . '/../public/swagger-ui.html');
$response->html($html, 200);
```

`Core/Router.php`

O que faz: Registra rotas e despacha requisições. Quando uma requisição chega, o Router encontra qual rota combina com o método + caminho e executa os middlewares e o handler correspondentes.

Registro de rotas:

```
// Registra uma rota GET sem middleware
$router->get('/clientes', ['ClienteController', 'listar']);

// Registra uma rota com parâmetro dinâmico e middleware
$router->get('/clientes/{id}', ['ClienteController', 'buscar'],
[AuthMiddleware::class]);

// Handler como closure (função anônima)
$router->get('/docs', function(Request $req, Response $res) {
    $res->html(file_get_contents('...'));
});
```

Como funciona o matching de rotas:

O Router converte o padrão `/clientes/{id}` em uma expressão regular:

```
/clientes/{id}  →  #^/clientes/(?P<id>[^/]+)$#
```

Quando a URL `/clientes/42` chega:

1. O regex faz match
2. `(?P<id>[^/]+)` captura `42` com o nome `id`
3. O Router injeta `['id' => '42']` na Request
4. O Controller lê via `$request->getParam('id')`

Fluxo de despacho:

```
// Router::despachar() — executado em public/index.php
Router::despachar(Request $request, Response $response): void

// Para cada rota registrada:
// 1. Verifica se o método HTTP bate
// 2. Faz match do caminho com regex
// 3. Injeta parâmetros de rota na Request
// 4. Executa middlewares em ordem
//    → Se algum retornar false, para e envia a resposta de erro
// 5. Executa o handler (Controller ou closure)
// 6. Chama $response->send()
```

`Database/Connection.php`

O que faz: Fornece a conexão PDO com o banco de dados. Implementa o padrão **Singleton** — garante que exista apenas uma instância de conexão durante toda a requisição.

Padrão Singleton:

```
class Connection {
    private static ?PDO $instancia = null;

    public static function getInstance(): PDO {
        if (self::$instancia === null) {
            // Cria a conexão apenas na primeira chamada
            $config = require __DIR__ . '/../../config/database.php';
            $dsn = "mysql:host={$config['host']};dbname=
{$config['dbname']};charset=utf8mb4";
            self::$instancia = new PDO($dsn, $config['usuario'], $config['senha'],
[
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ]);
        }
        return self::$instancia; // Retorna sempre a mesma instância
    }
}
```

Como é usado no projeto:

```
// ClienteModel.php — qualquer método que precisa do banco
class ClienteModel {
    private PDO $db;

    public function __construct() {
        $this->db = Connection::getInstance(); // pega a conexão única
    }

    public function buscarPorId(int $id): ?array {
        $stmt = $this->db->prepare('SELECT * FROM clientes WHERE id = ?');
        $stmt->execute([$id]);
        return $stmt->fetch() ?: null;
    }
}
```

Por que `ATTR_EMULATE_PREPARES = false`?

Por padrão, o PDO pode "simular" prepared statements sem enviá-los ao MySQL. Com `false`, os prepared statements reais são usados — o MySQL recebe o SQL e os parâmetros separadamente, impossibilitando SQL Injection mesmo com dados maliciosos.

Middleware/AuthMiddleware.php

O que faz: Intercepta requisições a rotas protegidas e verifica se o JWT é válido antes de permitir o acesso ao Controller.

Fluxo:

```
Requisição → AuthMiddleware → Controller
                 ↓
    Lê "Authorization: Bearer {token}"
                 ↓
    JWT::validar($token, $secret)
         ↙                ↘
    Válido               Inválido/Expirado
       ↓                     ↓
  injeta $payload        retorna false
  na Request             + 401 Unauthorized
       ↓
  retorna true
       ↓
  Controller executa
```

Como é usado no projeto:

```php
// routes/api.php — aplica middleware nas rotas de clientes
$router->get('/clientes', ['ClienteController', 'listar'],
[AuthMiddleware::class]);
$router->post('/clientes', ['ClienteController', 'criar'],
[AuthMiddleware::class]);
// etc.

// ClienteController.php — acessa os dados do usuário autenticado
// (injetados pelo middleware na Request)
$usuario = $request->getAtributo('usuario');
// $usuario = ['sub' => 1, 'usuario' => 'admin', 'exp' => 1700000900, ...]
```

Models/UsuarioModel.php

O que faz: Acessa os dados de usuários e refresh tokens no banco de dados.

Métodos principais:

```php
// Busca usuário por nome (para login)
buscarPorUsuario(string $usuario): ?array
// Retorna: ['id' => 1, 'usuario' => 'admin', 'senha' => '$2y$10$...'] ou null

// Salva refresh token no banco
```

```
salvarRefreshToken(int $usuarioId, string $token, string $expiraEm): void

// Busca refresh token para validação (JOIN com usuários)
buscarRefreshToken(string $token): ?array
// Retorna: dados do token + dados do usuário, ou null se expirado/inexistente

// Remove refresh token (invalida após uso — Rotation)
removerRefreshToken(string $token): void

// Remove todos os refresh tokens (logout de todos os dispositivos)
removerTodosRefreshTokens(int $usuarioId): void
```

Como é usado no projeto:

```
// AuthService.php — fluxo de login
$usuario = $this->usuarioModel->buscarPorUsuario('admin');
// verifica senha...
// gera tokens...
$expiraEm = date('Y-m-d H:i:s', time() + $config['expiracao_refresh']);
$this->usuarioModel->salvarRefreshToken($usuario['id'], $refreshToken, $expiraEm);

// AuthService.php — fluxo de refresh
$tokenData = $this->usuarioModel->buscarRefreshToken($refreshToken);
if (!$tokenData) { throw new InvalidArgumentException('Token inválido'); }
$this->usuarioModel->removerRefreshToken($refreshToken); // invalida o antigo
// gera novo par de tokens...
```

`Models/ClienteModel.php`

O que faz: CRUD completo de clientes no banco de dados.

Métodos:

```
listarTodos(): array              // SELECT * FROM clientes ORDER BY nome
buscarPorId(int $id): ?array      // SELECT * WHERE id = ?
buscarPorCpf(string $cpf): ?array // SELECT id, cpf WHERE cpf = ? (para verificar
duplicatas)
criar(array $dados): int          // INSERT + retorna lastInsertId()
atualizar(int $id, array $dados): bool  // UPDATE WHERE id = ?
excluir(int $id): bool            // DELETE WHERE id = ?
```

Exemplo — método `criar`:

```
// Input vindo do ClienteService após validação:
$dados = [
    'cpf'             => '52998224725',  // sem formatação
```

```
    'nome'            => 'Maria da Silva',
    'data_nascimento' => '1985-06-15',
    'whatsapp'        => '(11) 99999-9999',
    'email'           => 'maria@email.com',
];
$id = $this->clienteModel->criar($dados);
// Executa: INSERT INTO clientes (cpf, nome, data_nascimento, whatsapp, email)
//          VALUES (?, ?, ?, ?, ?)
// Retorna: 1 (o ID do cliente inserido)
```

Services/AuthService.php

O que faz: Toda a lógica de autenticação — verificar senha, gerar tokens, implementar refresh token rotation.

Métodos públicos:

```
login(string $usuario, string $senha): array
// Retorna: ['access_token' => '...', 'refresh_token' => '...', 'token_type' =>
'Bearer', 'expires_in' => 900]
// Lança: InvalidArgumentException se credenciais inválidas

renovarTokens(string $refreshToken): array
// Retorna: novo par de tokens
// Lança: InvalidArgumentException se token inválido/expirado
```

Fluxo detalhado do login:

```
// 1. Busca usuário no banco
$usuario = $this->usuarioModel->buscarPorUsuario($nomeUsuario);
if (!$usuario) { throw new InvalidArgumentException('Credenciais inválidas'); }

// 2. Verifica senha com bcrypt
if (!password_verify($senha, $usuario['senha'])) {
    throw new InvalidArgumentException('Credenciais inválidas');
}
// Nota: mesmo erro para usuário inexistente e senha errada
// (não revela se o usuário existe — segurança)

// 3. Gera e salva tokens
return $this->gerarTokens($usuario);
```

Services/ClienteService.php

O que faz: Lógica de negócio para gestão de clientes — validações, verificação de duplicatas, orquestração das operações.

Validações implementadas:

```
// CPF: formato + dígitos verificadores
validarCpf('529.982.247-25') // → true
validarCpf('111.111.111-11') // → false (todos iguais)
validarCpf('529.982.247-00') // → false (dígitos incorretos)

// Data: formato YYYY-MM-DD
validarData('1990-12-31') // → true
validarData('31/12/1990') // → false

// Email: formato válido (se fornecido)
// Nome: mínimo 3 caracteres
```

Como as validações são reportadas:

O método `validarDados()` **acumula todos os erros** antes de lançar a exceção. Isso significa que o cliente recebe todos os problemas de uma vez, não um por um:

```
// Resposta quando múltiplos campos são inválidos:
{
  "status": "error",
  "message": "CPF inválido. Nome deve ter pelo menos 3 caracteres. Data de
nascimento inválida."
}
```

`Controllers/AuthController.php`

O que faz: Recebe as requisições de autenticação, extrai os dados do corpo e delega ao AuthService.

Endpoints:

```
POST /auth/login
Body: { "usuario": "admin", "senha": "123456" }

Sucesso (200):
{
  "status": "success",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiJ9...",
    "refresh_token": "a1b2c3d4e5f6...",
    "token_type": "Bearer",
    "expires_in": 900
  }
}

Erro de campo faltando (400):
```

```
{ "status": "error", "message": "Campos obrigatórios: usuario, senha" }

Credenciais inválidas (401):
{ "status": "error", "message": "Credenciais inválidas" }
```

```
POST /auth/refresh
Body: { "refresh_token": "a1b2c3d4e5f6..." }

Sucesso (200): mesmo formato do login
Erro (401): { "status": "error", "message": "Refresh token inválido ou expirado" }
```

`Controllers/ClienteController.php`

O que faz: Gerencia as operações de CRUD de clientes. Todas as rotas são protegidas pelo `AuthMiddleware`.

Exemplo — fluxo de criação:

```
POST /clientes
Authorization: Bearer eyJ...
Content-Type: application/json

{
  "cpf": "529.982.247-25",
  "nome": "Maria da Silva",
  "data_nascimento": "1985-06-15",
  "whatsapp": "(11) 99999-9999",
  "email": "maria@email.com"
}

→ Controller verifica se campos obrigatórios estão presentes
→ Chama ClienteService::criar($dados)
→ Service valida CPF, nome, data
→ Service verifica se CPF já existe
→ Model insere no banco
→ Model busca o registro recém-criado
→ Service retorna o cliente completo
→ Controller retorna 201 com os dados

Resposta (201 Created):
{
  "status": "success",
  "data": {
    "id": 5,
    "cpf": "52998224725",
    "nome": "Maria da Silva",
    "data_nascimento": "1985-06-15",
    "whatsapp": "(11) 99999-9999",
    "email": "maria@email.com",
```

```
      "created_at": "2026-03-18 10:30:00",
      "updated_at": "2026-03-18 10:30:00"
    }
  }
```

---

5. Padrão Swagger e o swagger-ui.html

O que é Swagger / OpenAPI?

OpenAPI (anteriormente chamado Swagger) é um **padrão de especificação** para descrever APIs REST de forma legível tanto por humanos quanto por máquinas. A especificação é escrita em JSON ou YAML e descreve:

- Quais endpoints existem
- Quais métodos HTTP cada um aceita
- Quais parâmetros e body são esperados
- Quais respostas podem retornar (com exemplos)
- Como a autenticação funciona

Swagger UI é uma ferramenta que lê essa especificação e gera uma **interface web interativa**, onde você pode visualizar e testar a API direto no navegador.

```
docs/openapi.json        ←── Especificação (contrato da API)
        ↓
public/swagger-ui.html   ←── Interface visual que lê e exibe a spec
        ↓
Desenvolvedor/Estudante  ←── Explora e testa a API no navegador
```

Estrutura do `docs/openapi.json`

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "API REST Acadêmica",
    "version": "1.0.0",
    "description": "..."
  },
  "servers": [
    { "url": "http://localhost/a/v2/public", "description": "Desenvolvimento" }
  ],
  "components": {
    "securitySchemes": {
      "bearerAuth": {
        "type": "http",
        "scheme": "bearer",
        "bearerFormat": "JWT"
```

```
        }
      },
      "schemas": {
        "Cliente": { ... },
        "ClienteInput": { ... },
        "TokenResponse": { ... },
        "Erro": { ... }
      }
    },
    "paths": {
      "/auth/login": {
        "post": {
          "summary": "Realiza login",
          "requestBody": { ... },
          "responses": {
            "200": { "$ref": "#/components/schemas/TokenResponse" },
            "401": { "$ref": "#/components/schemas/Erro" }
          }
        }
      },
      "/clientes": {
        "get": {
          "security": [{ "bearerAuth": [] }],
          ...
        }
      }
    }
  }
```

Elementos principais:

Elemento	Descrição
`openapi`	Versão da especificação (3.0.3)
`info`	Metadados da API (título, versão, descrição)
`servers`	URLs base onde a API pode ser acessada
`paths`	Todos os endpoints e suas operações
`components/schemas`	Reutilização de schemas (evita repetição)
`components/securitySchemes`	Como a autenticação funciona
`security`	Quais endpoints requerem autenticação

Como o `swagger-ui.html` funciona neste projeto

O arquivo `public/swagger-ui.html` carrega o **Swagger UI via CDN** e o aponta para o arquivo `openapi.json` do projeto.

Funcionamento passo a passo:

```
1. Usuário acessa: http://localhost/a/v2/public/docs
2. routes/api.php registra a rota GET /docs → serve swagger-ui.html
3. O HTML carrega os assets do Swagger UI via CDN:
   - https://unpkg.com/swagger-ui-dist@5/swagger-ui.css
   - https://unpkg.com/swagger-ui-dist@5/swagger-ui-bundle.js
4. O JavaScript inicializa a interface:
   SwaggerUIBundle({
     url: "/docs/openapi.json",  ← aponta para a especificação
     dom_id: '#swagger-ui',
     ...
   });
5. Swagger UI faz GET /docs/openapi.json (outra rota registrada)
6. O JSON é lido e a interface é renderizada
```

Como usar o Swagger UI para testar:

1. Acesse `http://localhost/a/v2/public/docs`
2. Você verá todos os endpoints listados
3. Para rotas protegidas: a. Expanda `POST /auth/login` → clique "Try it out" b. Preencha `{"usuario": "admin", "senha": "123456"}` → Execute c. Copie o `access_token` da resposta d. Clique no botão **"Authorize"** (cadeado) no topo e. Cole: `Bearer eyJhbGciOiJIUzI1NiJ9...` f. Agora todas as requisições incluirão o token automaticamente

Vantagens do Swagger UI neste projeto:

- Documentação sempre sincronizada com o código (edita o JSON, muda a doc)
- Ambiente de testes integrado (sem precisar de Postman/Insomnia)
- Facilita onboarding de novos desenvolvedores
- Demonstra os schemas esperados de request/response

---

6. 50 Perguntas sobre o Projeto

Arquitetura e Estrutura

1. Qual padrão arquitetural é utilizado neste projeto e quais são suas três camadas principais?

2. Por que apenas a pasta `public/` é exposta ao servidor Apache, e não toda a pasta `v2/`?

3. Qual é a função do arquivo `public/index.php` e como ele se relaciona com o conceito de "Front Controller"?

4. O que é o arquivo `.htaccess` na raiz do projeto e qual sua diferença em relação ao `.htaccess` dentro de `public/`?

5. Por que existe uma pasta `routes/` separada? Qual o benefício de centralizar as definições de rotas?

6. Qual a diferença entre a pasta `app/Core/` e as demais pastas dentro de `app/`?

7. Por que os arquivos `.env` não devem ser commitados no repositório Git?

8. Qual é a responsabilidade da camada Service e por que ela existe separada do Controller?

9. O que é o padrão Singleton e onde ele é aplicado neste projeto?

10. Como o autoloader PSR-4 implementado no `index.php` associa um nome de classe ao seu arquivo?

Autenticação e Segurança

11. Qual a diferença entre um Access Token e um Refresh Token neste projeto?

12. O que é Refresh Token Rotation e por que esse mecanismo aumenta a segurança?

13. Por que as mensagens de erro de login retornam "Credenciais inválidas" tanto para usuário inexistente quanto para senha errada?

14. O que é um ataque de timing e como o método `hash_equals()` o previne?

15. Quais são as três partes de um JWT e o que cada uma contém?

16. Por que o payload de um JWT é Base64Url encoded e não criptografado? Qual implicação isso tem?

17. Qual a diferença entre autenticação e autorização?

18. O que são prepared statements e como eles previnem SQL Injection?

19. Qual atributo PDO garante que os prepared statements sejam executados de forma real no MySQL (não simulada)?

20. Como o `AuthMiddleware` injeta os dados do usuário autenticado para que o Controller possa acessá-los?

JWT e Tokens

21. Quais são os claims padrão (standard claims) utilizados no payload JWT deste projeto?

22. Se o `JWT_ACCESS_EXPIRY` está configurado como `900`, por quanto tempo o access token é válido?

23. Onde os refresh tokens são armazenados e por que não são armazenados no cliente (diferente do access token)?

24. O que acontece com o refresh token antigo quando o cliente faz uma requisição para `/auth/refresh`?

25. Por que o algoritmo HS256 usa uma chave secreta (`JWT_SECRET`) e qual o risco de essa chave ser fraca ou vazar?

Rotas e Controllers

26. Como o Router converte o padrão de rota `/clientes/{id}` em uma expressão regular?

27. Quais são os códigos HTTP retornados pelo `ClienteController` e em quais situações cada um é usado?

28. O que acontece quando o Router não encontra nenhuma rota que corresponda à requisição?

29. Qual a diferença entre registrar um handler como `['ClienteController', 'listar']` e como uma closure/função anônima?

30. Por que a rota `DELETE /clientes/{id}` não precisa de um corpo (body) na requisição?

Validações e Modelos

31. Quais campos são obrigatórios para criar um cliente e quais são opcionais?

32. Explique o algoritmo de validação de CPF implementado em `ClienteService`. O que são os dígitos verificadores?

33. Por que o método `validarDados()` acumula todos os erros antes de lançar a exceção, ao invés de lançar na primeira falha?

34. Como o projeto verifica se um CPF já está cadastrado ao criar ou atualizar um cliente?

35. Qual é o formato esperado para `data_nascimento` e por que esse formato específico é utilizado?

Banco de Dados

36. Quais são as três tabelas do banco de dados e qual a relação entre elas?

37. Por que a tabela `refresh_tokens` tem uma `FOREIGN KEY` com `ON DELETE CASCADE` para a tabela `usuarios`?

38. Por que o CPF é armazenado sem formatação (apenas números) no banco de dados?

39. O que faz o campo `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP` na tabela de clientes?

40. Para que serve o arquivo `database/seed.php` e quando ele deve ser executado?

OpenAPI e Swagger

41. Qual é a diferença entre OpenAPI e Swagger UI?

42. Para que serve o campo `security: [{ "bearerAuth": [] }]` em um endpoint na especificação OpenAPI?

43. O que é o `$ref` em um arquivo OpenAPI e qual problema ele resolve?

44. Como o `swagger-ui.html` obtém a especificação OpenAPI para renderizar a interface?

45. Por que o Swagger UI é uma ferramenta útil mesmo para desenvolvedores que já conhecem a API?

CORS e HTTP

46. O que é CORS e por que o projeto configura o cabeçalho `Access-Control-Allow-Origin: *`?

47. Por que o `index.php` verifica se o método da requisição é `OPTIONS` e retorna imediatamente com status 200?

48. Qual a diferença semântica entre os códigos HTTP 200, 201, 400, 401, 404 e 409?

49. O que é o cabeçalho `Authorization: Bearer {token}` e como ele se diferencia do `Authorization: Basic {credentials}`?

50. Por que o projeto usa `Content-Type: application/json` em todas as respostas e qual a importância disso para os clientes HTTP?

---

7. Gabarito Comentado

Arquitetura e Estrutura

1. Qual padrão arquitetural é utilizado neste projeto e quais são suas três camadas principais?

Resposta: O padrão **MVC** (Model-View-Controller). Neste projeto as três camadas são: **Model** (acesso ao banco — `app/Models/`), **Controller** (recebe requisições e monta respostas — `app/Controllers/`) e, no lugar de View, usa-se **Service** (lógica de negócio — `app/Services/`). Como é uma API REST, não há View no sentido tradicional (HTML renderizado); o JSON retornado é o equivalente.

> Em APIs REST é comum adaptar o MVC para um padrão **MCS** (Model-Controller-Service), já que não há interface visual para renderizar.

---

2. Por que apenas a pasta `public/` é exposta ao servidor Apache, e não toda a pasta `v2/`?

Resposta: Por **segurança**. Se toda a pasta fosse exposta, um atacante poderia acessar diretamente arquivos sensíveis como `.env` (com credenciais do banco e chave JWT), arquivos de configuração (`config/database.php`) e código-fonte dos models e services. Ao expor apenas `public/`, somente o que foi intencionalmente colocado ali é acessível via HTTP.

> Esta é uma prática padrão em qualquer framework PHP profissional: Laravel usa `public/`, Symfony usa `public/`, CodeIgniter usa `public/`.

---

3. Qual é a função do arquivo `public/index.php` e como ele se relaciona com o conceito de "Front Controller"?

Resposta: O `index.php` é o **Front Controller** — um ponto de entrada único para todas as requisições da aplicação. O padrão Front Controller centraliza o bootstrapping (carregamento de autoloader, variáveis de ambiente, CORS, tratamento de erros) e garante que toda requisição passe pelo mesmo pipeline antes de chegar ao código específico de cada rota.

> Sem Front Controller, cada arquivo PHP seria acessado diretamente, obrigando a duplicar o código de inicialização em todos eles.

---

4. O que é o arquivo `.htaccess` na raiz do projeto e qual sua diferença em relação ao `.htaccess` dentro de `public/`?

Resposta: O `.htaccess` na **raiz** redireciona toda requisição para a pasta `public/` (garante que o Apache sirva a partir de `public/`). O `.htaccess` dentro de **`public/`** redireciona toda requisição que não seja um arquivo ou diretório existente para `index.php` — isso é o que permite URLs limpas como `/clientes/5` sem que o Apache procure um arquivo chamado `5` dentro de uma pasta `clientes`.

---

5. Por que existe uma pasta `routes/` separada? Qual o benefício de centralizar as definições de rotas?

Resposta: Separar as rotas segue o **Princípio da Responsabilidade Única (SRP)**. Com todas as rotas em `routes/api.php`, é possível visualizar toda a API de uma só vez — quais URLs existem, quais métodos aceitam, quais controllers respondem e quais middlewares protegem cada rota. Isso facilita manutenção, onboarding de novos devs e evita que a lógica de roteamento se misture com a de bootstrapping (`index.php`).

---

6. Qual a diferença entre a pasta `app/Core/` e as demais pastas dentro de `app/`?

Resposta: `app/Core/` contém infraestrutura genérica do micro-framework (`Router`, `Request`, `Response`, `JWT`, `Env`) que **não pertence ao domínio do negócio**. Poderia ser reutilizada em qualquer projeto PHP. As demais pastas (`Controllers`, `Services`, `Models`, `Middleware`) contêm código específico do domínio desta aplicação (clientes, autenticação).

---

7. Por que os arquivos `.env` não devem ser commitados no repositório Git?

Resposta: Porque `.env` contém **credenciais sensíveis**: usuário e senha do banco de dados, chave secreta JWT, configurações de ambiente. Se committado, essas informações ficam acessíveis a qualquer pessoa com acesso ao repositório (incluindo histórico Git). O padrão é commitar apenas `.env.example` (com valores de exemplo/vazios) e adicionar `.env` ao `.gitignore`.

---

8. Qual a responsabilidade da camada Service e por que ela existe separada do Controller?

Resposta: A camada Service contém a **lógica de negócio** — regras que fazem a aplicação ter valor: validar CPF, garantir unicidade, implementar refresh token rotation. Separar do Controller tem duas vantagens principais: **reutilização** (a mesma lógica pode ser chamada de múltiplos controllers, comandos CLI ou filas) e **testabilidade** (Services podem ser testados unitariamente sem simular requisições HTTP).

---

9. O que é o padrão Singleton e onde ele é aplicado neste projeto?

Resposta: Singleton é um padrão de projeto que garante que uma classe tenha **apenas uma instância** durante toda a execução. É aplicado em `Database/Connection.php`: a primeira chamada a `Connection::getInstance()` cria a conexão PDO; chamadas subsequentes retornam a mesma instância já criada. Isso evita abrir múltiplas conexões com o banco em uma única requisição, o que seria custoso em recursos.

---

10. Como o autoloader PSR-4 implementado no `index.php` associa um nome de classe ao seu arquivo?

Resposta: O autoloader mapeia **namespaces/prefixos para diretórios**. Quando o PHP precisa de uma classe (ex: `ClienteController`), chama o autoloader que consulta o mapa:

- `Controllers\` → `app/Controllers/`
- `Services\` → `app/Services/`
- `Models\` → `app/Models/` etc.

Substitui `\` por `/`, adiciona `.php` e tenta incluir o arquivo. PSR-4 é o padrão da comunidade PHP para autoloading (usado pelo Composer).

Autenticação e Segurança

11. Qual a diferença entre um Access Token e um Refresh Token neste projeto?

Resposta: O **Access Token** é um JWT de curta duração (15 minutos) enviado no cabeçalho `Authorization` de cada requisição protegida. O servidor valida sua assinatura sem consultar o banco. O **Refresh Token** é uma string aleatória de longa duração (7 dias) armazenada no banco de dados, usada apenas para obter um novo par de tokens quando o access token expira. O cliente armazena o refresh token de forma segura (nunca expõe em cada requisição).

---

12. O que é Refresh Token Rotation e por que esse mecanismo aumenta a segurança?

Resposta: Refresh Token Rotation significa que, a cada uso do refresh token, ele é **invalidado** e um novo é gerado. Se um atacante roubar o refresh token e tentar usá-lo *depois* do usuário legítimo já ter feito refresh, o servidor detecta que o token foi usado mais de uma vez — indicando comprometimento. Isso limita a janela de exposição: um token roubado é inútil após o uso legítimo.

---

13. Por que as mensagens de erro de login retornam "Credenciais inválidas" tanto para usuário inexistente quanto para senha errada?

Resposta: Para prevenir **user enumeration** — um atacante não deve conseguir distinguir se um usuário existe ou não no sistema. Se a API retornasse "Usuário não encontrado" vs "Senha incorreta", um atacante poderia fazer um ataque de dicionário apenas para descobrir quais usuários existem, e então focar apenas neles para tentar as senhas.

---

14. O que é um ataque de timing e como o método `hash_equals()` o previne?

Resposta: Um ataque de timing explora o fato de que comparações de string normais (`===`) podem ser interrompidas mais cedo quando encontram um caractere diferente — strings mais parecidas levam mais tempo para falhar. Medindo o tempo de resposta, um atacante pode deduzir quão "próxima" sua tentativa está do valor correto. `hash_equals()` sempre compara as strings inteiras, levando o mesmo tempo independentemente das diferenças, tornando esse ataque inviável.

---

15. Quais são as três partes de um JWT e o que cada uma contém?

Resposta:

- **Header** (Base64Url): tipo do token (`JWT`) e algoritmo de assinatura (`HS256`)
- **Payload** (Base64Url): claims — dados do usuário e metadados como `iss` (emissor), `sub` (sujeito/ID), `exp` (expiração), `iat` (emitido em), `usuario` (nome)
- **Signature**: resultado de `HMAC-SHA256(base64url(header) + "." + base64url(payload), secret)` — prova que o token não foi adulterado

---

16. Por que o payload de um JWT é Base64Url encoded e não criptografado? Qual implicação isso tem?

Resposta: JWT padrão (JWS) é **assinado, não criptografado**. O Base64Url é apenas uma codificação — qualquer pessoa pode decodificar e ler o payload. A implicação prática: **nunca coloque dados sensíveis no**

payload (senhas, dados financeiros, informações pessoais confidenciais). O JWT garante que o payload não foi adulterado (a assinatura), mas não que está oculto.

---

17. Qual a diferença entre autenticação e autorização?

Resposta: Autenticação ("quem você é") verifica a identidade — o login com usuário/senha que gera o JWT. **Autorização** ("o que você pode fazer") controla o acesso a recursos após a identidade ser confirmada — o `AuthMiddleware` verifica se o token é válido antes de permitir acesso aos endpoints de clientes. Este projeto implementa ambas, mas a autorização é simples: ou tem token válido ou não tem.

---

18. O que são prepared statements e como eles previnem SQL Injection?

Resposta: Prepared statements separam o **código SQL** dos **dados**. O SQL é enviado primeiro ao MySQL (que o compila/prepara), e depois os dados são enviados separadamente como parâmetros. O MySQL trata os parâmetros sempre como dados — nunca como código SQL. Assim, mesmo que o dado contenha `'; DROP TABLE clientes; --`, ele será tratado como uma string literal e não como instrução SQL.

---

19. Qual atributo PDO garante que os prepared statements sejam executados de forma real no MySQL (não simulada)?

Resposta: `PDO::ATTR_EMULATE_PREPARES => false`. Por padrão, o PDO pode simular prepared statements no lado PHP (interpolando os parâmetros antes de enviar ao MySQL), o que pode ter vulnerabilidades. Com `false`, os prepared statements reais são usados — o SQL e os parâmetros chegam ao MySQL separadamente, eliminando qualquer risco de injeção.

---

20. Como o `AuthMiddleware` injeta os dados do usuário autenticado para que o Controller possa acessá-los?

Resposta: O `AuthMiddleware` valida o JWT e chama `$request->setAtributo('usuario', $payload)`, onde `$payload` é o array com os claims decodificados. O objeto `$request` é passado por referência pela cadeia de execução, então quando o Controller recebe a requisição, pode chamar `$request->getAtributo('usuario')` e obter os dados do usuário sem precisar revalidar o token.

JWT e Tokens

21. Quais são os claims padrão (standard claims) utilizados no payload JWT deste projeto?

Resposta:

- `iss` (issuer): `"api-academica"` — identifica quem emitiu o token
- `iat` (issued at): timestamp Unix de quando foi emitido
- `exp` (expiration): timestamp Unix de quando expira
- `sub` (subject): ID do usuário (chave primária)
- `usuario`: nome de login (claim customizado)

---

22. Se o `JWT_ACCESS_EXPIRY` está configurado como `900`, por quanto tempo o access token é válido?

Resposta: 15 minutos. `900 segundos ÷ 60 = 15 minutos`. O timestamp de expiração no payload é calculado como `time() + 900`, onde `time()` retorna o Unix timestamp atual (segundos desde 01/01/1970). Ao validar, o JWT compara `exp` com `time()` — se `exp < time()`, o token está expirado.

---

23. Onde os refresh tokens são armazenados e por que não são armazenados no cliente (diferente do access token)?

Resposta: Os refresh tokens são armazenados na **tabela `refresh_tokens` do banco de dados**. Ao contrário do access token (JWT auto-contido, validado apenas pela assinatura), o refresh token precisa ser **revogável** — ao fazer logout, ao detectar comprometimento ou ao implementar rotation, o servidor precisa poder invalidá-lo. Se fosse apenas no cliente (como o JWT), não haveria como invalidá-lo antes de sua expiração natural.

---

24. O que acontece com o refresh token antigo quando o cliente faz uma requisição para `/auth/refresh`?

Resposta: O refresh token antigo é **imediatamente excluído** do banco (`removerRefreshToken($refreshToken)`) antes de gerar o novo par. Isso é a essência do Refresh Token Rotation: cada token pode ser usado **exatamente uma vez**. Se um atacante tentar usar um refresh token após o usuário legítimo já ter feito refresh, o token não existirá mais no banco e a requisição será rejeitada com 401.

---

25. Por que o algoritmo HS256 usa uma chave secreta (`JWT_SECRET`) e qual o risco de essa chave ser fraca ou vazar?

Resposta: HS256 é **HMAC-SHA256** — um algoritmo simétrico que usa a mesma chave para assinar e verificar. Se `JWT_SECRET` for fraca (curta, previsível), um atacante pode fazer brute force para descobri-la. Se vazar, o atacante pode gerar tokens válidos para qualquer usuário. Por isso a recomendação é usar uma string aleatória longa (64+ caracteres) e nunca expô-la. Em produção, deve ser diferente da de desenvolvimento.

Rotas e Controllers

26. Como o Router converte o padrão de rota `/clientes/{id}` em uma expressão regular?

Resposta: O Router substitui cada segmento `{nome}` pelo padrão de regex `(?P<nome>[^/]+)`, que captura qualquer caractere exceto `/`, nomeando o grupo de captura. O padrão completo é encapsulado com delimitadores e âncoras:

```
/clientes/{id}  →  #^/clientes/(?P<id>[^/]+)$#
```

Quando a URL `/clientes/42` faz match, o grupo `id` captura `42`, que é injetado em `$request->parametrosRota['id']`.

---

27. Quais são os códigos HTTP retornados pelo `ClienteController` e em quais situações cada um é usado?

Resposta:

- **200 OK**: operações bem-sucedidas (listar, buscar, atualizar, excluir)
- **201 Created**: cliente criado com sucesso
- **400 Bad Request**: dados inválidos (CPF inválido, campos obrigatórios faltando)
- **401 Unauthorized**: token JWT inválido ou ausente (retornado pelo middleware)
- **404 Not Found**: cliente com o ID informado não existe
- **409 Conflict**: CPF já cadastrado (unicidade violada)

---

28. O que acontece quando o Router não encontra nenhuma rota que corresponda à requisição?

Resposta: O Router retorna uma resposta **404 Not Found** com corpo JSON:

```
{ "status": "error", "message": "Rota não encontrada" }
```

Isso é feito ao final do método `despachar()`, após percorrer todas as rotas sem encontrar match. É importante que seja um JSON e não uma página HTML de erro do PHP/Apache, já que a API deve ser consistente em seu formato de resposta.

---

29. Qual a diferença entre registrar um handler como `['ClienteController', 'listar']` e como uma closure?

Resposta: O handler como array `['ClienteController', 'listar']` instrui o Router a instanciar a classe `ClienteController` e chamar seu método `listar`. É o formato padrão para controllers — permite organização em classes e reutilização de lógica no construtor. A closure (função anônima) é útil para rotas simples que não justificam um controller inteiro — como servir o HTML do Swagger ou retornar informações estáticas.

---

30. Por que a rota `DELETE /clientes/{id}` não precisa de um corpo (body) na requisição?

Resposta: Porque toda a informação necessária para a operação está na **URL**: o ID do cliente a ser excluído. O verbo `DELETE` semanticamente significa "exclua o recurso identificado por esta URL". Enviar um body em DELETE é tecnicamente permitido pelo HTTP, mas é incomum e desnecessário quando o identificador está na URL.

Validações e Modelos

31. Quais campos são obrigatórios para criar um cliente e quais são opcionais?

Resposta:

- **Obrigatórios**: `cpf`, `nome`, `data_nascimento`
- **Opcionais**: `whatsapp`, `email`

Os campos opcionais são armazenados como `NULL` no banco quando não fornecidos. Mesmo opcionais, se fornecidos, passam por validação (o email precisa ser um endereço válido).

---

32. Explique o algoritmo de validação de CPF. O que são os dígitos verificadores?

Resposta: Um CPF tem 11 dígitos: os 9 primeiros são o número base e os 2 últimos são **dígitos verificadores**, calculados a partir dos 9 primeiros. O algoritmo:

1. Multiplica os 9 primeiros dígitos pelos pesos 10 a 2, soma os produtos
2. Calcula o resto da divisão por 11; se resto < 2, primeiro dígito = 0; senão = 11 - resto
3. Repete o processo para os 10 primeiros dígitos com pesos 11 a 2 para obter o segundo dígito

Se os dígitos calculados baterem com os informados, o CPF é matematicamente válido. CPFs com todos os dígitos iguais (111.111.111-11) são rejeitados mesmo passando pelo algoritmo.

---

33. Por que o método `validarDados()` acumula todos os erros antes de lançar a exceção?

Resposta: Para melhorar a **experiência do usuário/desenvolvedor**. Se a validação parasse na primeira falha, o cliente precisaria corrigir e reenviar a requisição múltiplas vezes para descobrir todos os problemas. Acumulando os erros em um array e retornando todos de uma vez, o cliente recebe feedback completo em uma única interação. É o comportamento esperado em APIs bem projetadas.

---

34. Como o projeto verifica se um CPF já está cadastrado ao criar ou atualizar um cliente?

Resposta: O `ClienteService` chama `ClienteModel::buscarPorCpf($cpf)` antes de inserir ou atualizar. Na **criação**, se retornar qualquer registro, lança `RuntimeException` com código 409. Na **atualização**, a verificação exclui o próprio cliente da busca (um cliente pode manter o mesmo CPF ao ser atualizado, o conflito só existe se outro cliente já tem o CPF).

---

35. Qual é o formato esperado para `data_nascimento` e por que esse formato específico é utilizado?

Resposta: O formato é `YYYY-MM-DD` (ex: `1990-12-31`). É o formato **ISO 8601**, padrão internacional para datas. É também o formato nativo do tipo `DATE` do MySQL, facilitando o armazenamento e a ordenação. Evita ambiguidade entre formatos regionais como `31/12/1990` (Brasil), `12/31/1990` (EUA) ou `31-12-1990`.

Banco de Dados

36. Quais são as três tabelas do banco de dados e qual a relação entre elas?

Resposta:

- `usuarios`: armazena usuários do sistema (id, usuario, senha)
- `refresh_tokens`: armazena refresh tokens ativos, com FK para `usuarios` (um usuário pode ter múltiplos tokens — vários dispositivos)
- `clientes`: armazena os clientes gerenciados pela API (sem relação direta com usuários — qualquer usuário autenticado pode gerenciar qualquer cliente)

Relação: `usuarios` 1→N `refresh_tokens` (um usuário, muitos tokens).

---

37. Por que a tabela `refresh_tokens` tem `ON DELETE CASCADE` para `usuarios`?

Resposta: Com `ON DELETE CASCADE`, ao excluir um usuário da tabela `usuarios`, todos os seus refresh tokens na tabela `refresh_tokens` são automaticamente excluídos pelo MySQL. Isso evita **dados órfãos** — tokens que apontam para um usuário inexistente. Sem CASCADE, a exclusão do usuário falharia com erro de violação de chave estrangeira (a menos que os tokens fossem deletados manualmente antes).

---

38. Por que o CPF é armazenado sem formatação (apenas números) no banco de dados?

Resposta: Para **consistência e eficiência**. Armazenar `52998224725` (apenas dígitos) ao invés de `529.982.247-25` (com formatação) garante que a busca por CPF seja simples (`WHERE cpf = ?`), o índice UNIQUE funcione corretamente e não haja ambiguidade de formatação. A formatação para exibição é responsabilidade da camada de apresentação (frontend), não do banco.

---

39. O que faz o campo `updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`?

Resposta: É um timestamp automático que o MySQL atualiza automaticamente para o momento atual sempre que qualquer campo do registro for alterado. Não é necessário incluí-lo explicitamente nas queries UPDATE — o MySQL o gerencia automaticamente. Serve para auditoria (quando o registro foi modificado pela última vez).

---

40. Para que serve o arquivo `database/seed.php` e quando ele deve ser executado?

Resposta: O seed popula o banco com dados iniciais necessários para que a aplicação funcione. Neste projeto, cria o usuário `admin` (sem o qual ninguém conseguiria fazer login) e alguns clientes de exemplo para demonstração. É executado **uma vez**, após criar o banco e rodar as migrations, com `php database/seed.php`. Em produção, apenas o usuário admin seria criado (não os clientes de exemplo).

OpenAPI e Swagger

41. Qual é a diferença entre OpenAPI e Swagger UI?

Resposta: OpenAPI é a **especificação** — um padrão (JSON/YAML) para descrever APIs REST de forma estruturada e legível por máquinas. **Swagger UI** é uma **ferramenta** (interface web) que lê uma especificação OpenAPI e a renderiza como documentação interativa. "Swagger" era o nome original da especificação, que foi doada à OpenAPI Initiative em 2016; Swagger UI continuou como nome da ferramenta visual.

---

42. Para que serve o campo `security: [{ "bearerAuth": [] }]` em um endpoint na especificação OpenAPI?

Resposta: Indica que aquele endpoint **requer autenticação** usando o esquema `bearerAuth` definido em `components/securitySchemes`. O Swagger UI usa essa informação para: exibir um cadeado no endpoint, incluir automaticamente o token JWT nas requisições de teste quando o usuário clicou em "Authorize" e inseriu seu token, e documentar claramente para os consumidores da API quais endpoints são protegidos.

---

43. O que é o `$ref` em um arquivo OpenAPI e qual problema ele resolve?

Resposta: `$ref` é uma referência para reutilizar definições em outros locais da especificação. Por exemplo, `"$ref": "#/components/schemas/Cliente"` reutiliza o schema `Cliente` definido em `components`. Isso resolve o problema de **duplicação** — sem `$ref`, o schema de um cliente precisaria ser repetido em cada endpoint que retorna clientes. Com `$ref`, define-se uma vez e referencia-se onde necessário.

---

44. Como o `swagger-ui.html` obtém a especificação OpenAPI para renderizar a interface?

Resposta: O `swagger-ui.html` inicializa o Swagger UI JavaScript com `url: "/docs/openapi.json"`. O Swagger UI então faz uma requisição HTTP GET para esse caminho, que está mapeado em `routes/api.php` para retornar o conteúdo do arquivo `docs/openapi.json`. O JavaScript processa o JSON recebido e renderiza toda a interface dinamicamente no navegador.

---

45. Por que o Swagger UI é uma ferramenta útil mesmo para desenvolvedores que já conhecem a API?

Resposta: Porque oferece um **ambiente de testes integrado** sem precisar configurar Postman ou escrever cURL. Permite testar rapidamente se um endpoint está funcionando, experimentar diferentes payloads, verificar o comportamento de casos de erro e demonstrar a API para outros stakeholders sem precisar de ferramentas externas. Também serve como documentação sempre atualizada quando a spec é mantida junto ao código.

CORS e HTTP

46. O que é CORS e por que o projeto configura `Access-Control-Allow-Origin: *`?

Resposta: CORS (Cross-Origin Resource Sharing) é um mecanismo de segurança do navegador que bloqueia requisições JavaScript para domínios diferentes do domínio da página. Por exemplo, um frontend em `http://meusite.com` não pode fazer fetch para `http://api.meusite.com` sem que a API declare explicitamente que permite isso. `Access-Control-Allow-Origin: *` permite requisições de **qualquer domínio** — adequado para APIs públicas ou em desenvolvimento. Em produção, o ideal é especificar os domínios permitidos.

---

47. Por que o `index.php` verifica se o método é `OPTIONS` e retorna 200 imediatamente?

Resposta: Antes de uma requisição "não simples" (POST com JSON, PUT, DELETE), o navegador envia automaticamente uma **preflight request** com método `OPTIONS` para verificar se o servidor permite a requisição real. Se o servidor não responder corretamente ao OPTIONS, o navegador bloqueia a requisição real. Retornar 200 com os cabeçalhos CORS corretos no OPTIONS autoriza o navegador a prosseguir com a requisição original.

---

48. Qual a diferença semântica entre os códigos HTTP 200, 201, 400, 401, 404 e 409?

Resposta:

- **200 OK**: requisição bem-sucedida, retorna dados
- **201 Created**: recurso criado com sucesso (resposta a POST)
- **400 Bad Request**: dados inválidos enviados pelo cliente (problema no input)

- **401 Unauthorized**: não autenticado — precisa de token válido
- **404 Not Found**: recurso não existe (cliente com aquele ID não existe)
- **409 Conflict**: conflito de estado — o CPF já está cadastrado, não é possível criar duplicata

---

49. Qual a diferença entre `Authorization: Bearer {token}` e `Authorization: Basic {credentials}`?

Resposta: Bearer carrega um token opaco (JWT neste caso) que o servidor decodifica para identificar o usuário — stateless, sem estado no servidor. **Basic** carrega `usuario:senha` codificados em Base64 — as credenciais completas são enviadas em cada requisição. Bearer é mais seguro para APIs (token tem expiração, pode ser revogado, não expõe a senha) enquanto Basic é mais simples mas envia as credenciais primárias repetidamente.

---

50. Por que o projeto usa `Content-Type: application/json` em todas as respostas e qual a importância disso para os clientes HTTP?

Resposta: O `Content-Type` diz ao cliente **como interpretar o corpo da resposta**. `application/json` instrui o cliente (navegador, Postman, fetch, cURL) a tratar o corpo como JSON. Sem esse cabeçalho, alguns clientes podem não parsear automaticamente a resposta ou interpretar incorretamente. Em JavaScript, `fetch()` usa o `Content-Type` para decidir se pode chamar `.json()` automaticamente. É uma parte essencial do contrato de uma API REST.

Apostila gerada em 18/03/2026 — API REST Acadêmica v2